

LAB#10

INDEXING IN MONGODB

- To learn and implement different types of indexes in MongoDB

Theory:

DEFINITION OF INDEXING IN DATABASES:

Indexing is a technique used to optimize the performance of a database by reducing the time it takes to retrieve data. In MongoDB, indexes are data structures that store a small portion of the data set in an easy-to-traverse form. This allows MongoDB to quickly locate and return documents.

IMPORTANCE OF INDEXING IN MONGODB:

Indexing plays a crucial role in improving query performance, especially when dealing with large datasets. By creating indexes on fields frequently used in queries, MongoDB can efficiently locate and retrieve documents, minimizing the need for scanning the entire collection.

Example:

We have a database bookstore with the collection 'books' and it contains following data.

```
> db.books.find()
< {
  _id: ObjectId('6594725c0dea87f373784298'),
  title: 'The Great Gatsby',
  author: 'F. Scott Fitzgerald',
  genre: 'Classic',
  price: 10.99,
  published_date: 1925-04-10T00:00:00.000Z,
  rating: 4
}
{
  _id: ObjectId('6594725c0dea87f373784299'),
  title: 'To Kill a Mockingbird',
  author: 'Harper Lee',
  genre: 'Fiction',
  price: 12.99,
  published_date: 1960-07-11T00:00:00.000Z,
  rating: 5
}
{
  _id: ObjectId('6594725c0dea87f37378429a'),
  title: '1984',
  author: 'George Orwell',
  genre: 'Dystopian',
  price: 9.99,
  published_date: 1949-06-08T00:00:00.000Z,
  rating: 4
}
```

Let's retrieve the books with the rating 4

```
db.books.find({ rating: 4 }).explain("executionStats")
```

```
executionStats: {  
  executionSuccess: true,  
  nReturned: 2,  
  executionTimeMillis: 55,  
  totalKeysExamined: 0,  
  totalDocsExamined: 3,  
  executionStages: {  
    stage: 'filter',  
    planNodeId: 1,  
    nReturned: 2,  
  },  
}
```

Here, as we can see that the query is returning 2 records with the rating 4. But the number of documents examined is 3 (Total records in the collection).

Now, Let's create index on the field 'rating'.

To create an index, we use the method **createIndex()**

To display the indexes on the collection, use the method **getIndexes()**.

```
> // Create an index on the "rating" field
db.books.createIndex({ rating: 1 })
< rating_1
> db.books.getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { rating: 1 }, name: 'rating_1' }
]
```

Now, let's run the same query and observe the results.

```
> // Explain and query books with a rating of 4
db.books.find({ rating: 4 }).explain("executionStats")
```

```

executionStats: {
  executionSuccess: true,
  nReturned: 2,
  executionTimeMillis: 160,
  totalKeysExamined: 2,
  totalDocsExamined: 2,
  executionStages: {
    stage: 'n1j',
    planNodeId: 2,
    nReturned: 2,
    executionTimeMillisEstimate: 38,
  }
}

```

Here, we can see that the number of examined documents has now decreased to two, which results in fast querying results.

TYPES OF INDEXES:

MongoDB provides different types of indexes that are used according to the data type or queries. The indexes supported by MongoDB are as follows:

1. Single field Index: A single field index means index on a single field of a document. This index is helpful for fetching data in ascending as well as descending order.

Syntax:

```
db.students.createIndex({"<fieldName>": <1 or -1>});
```

Here 1 represents the field is specified in ascending order and -1 for descending order.

Example: The general example we covered above can be used as the example for single field index

2. Compound Index: We can combine multiple fields for compound indexing and that will help for searching or filtering documents in that way. Or in other words, the compound index is an index where a single index structure holds multiple references.

Syntax:

```
db.<collection>.createIndex( { <field1>: <type>, <field2>: <type2>, ... } )
```

Here, we can combine the required fields in this pattern. Also the value of these fields is 1(for ascending order) or -1(for descending order).

Example:

Here, we have created a compound index on the field 'rating' and 'genre'.

```
db.books.createIndex({ rating: 1, genre: 1 })  
< rating_1_genre_1
```

3. Text Index

MongoDB supports query operations that perform a text search of string content. Text index allows us to find the string content in the specified collection. It can include any field that contains string content or an array of string items. A collection can contain at most one text index. You are allowed to use text index in the compound index.

Syntax:

db.<collection>.createIndex({ <field>: "text" })

We can give exact phrases also for searching by enclosing the search terms in double quotes

Example:

Text index created on the field Genre

```
db.books.createIndex({ genre: "text" })
```

Syntax for searching after text indexing.

```
db.books.find({ $text: { $search: "Classic" } })
```

LAB TASKS:

1. Create a collection named "products" and insert at least 10 documents.
2. Create a compound index on the "category" and "price" fields. Write a query that utilizes the index created.
3. Write a query that utilizes the newly created index to find products in a specific category with prices greater than a certain value.
(Note: Explore the implementation of for loop, keyword 'const' and embed it in this query result.)
4. Create a text index on the field category.
5. Create an index with key -1 and observe the difference.
6. Create a collection named "articles" with documents containing "title" and "content" fields. Create a text index on the "content" field. Perform a text search to find articles containing a specific keyword.
(Note: Explore the implementation of for loop, keyword 'const' and embed it in this query result.)

